

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

2. Error detection and correction

Error Detection

Error detection is used on the network to detect errors during transmission. Error detection codes are widely used on wireless or unguided networks, as wireless media have no physical shield, increasing the potential for errors such as data loss or corrupted data.

- Error detection is the situation when the received message does not match on the receiver side with the message sent by the sender.
- If we do not use error detection codes, it becomes very difficult to detect errors and resend data lost during transmission.
- In general, error detection and retransmission of data are usually more efficient than occasional error handling.

There are 3 types of error-detection codes we can use to detect errors in communication. They are as follows:

1. Parity Bit
2. Checksum
3. Cyclic Redundancy Check (CRC)

1. Parity bit

Parity bits and checksums are methods of detecting errors during transmission to ensure that data is not lost or corrupted. In this section, we will discuss both methods one by one.

Parity Bit: In the Parity bit method, an additional bit is added to each word to make error detection possible on the communication channel. The parity bit has two values, either 0 or 1.

Even Parity and odd parity are types of parity.

- **Even Parity:** The sender is sending the message as a stream of bits over the communication channel. After adding a parity bit (1 or 0) to the message, the number of 1's becomes even then it is known as even-parity.
- **Odd Parity:** The sender is sending the message as a stream of bits over the communication channel. After adding a parity bit (1 or 0) to the message, the number of 1's becomes odd then it is known as odd-parity.

Example code:

```
#include <stdio.h>
```

```
// Function to calculate even parity for a byte
int calculateParity(int data[], int size) {
    int parity = 0;
    for (int i = 0; i < size; i++) {
        parity ^= data[i];
    }
    return parity;
}

int main() {
    int data[8];
    int size = 8;

    // Input 8-bit data from the user
    printf("Enter 8-bit data (1 or 0 for each bit):\n");
    for (int i = 0; i < size; i++) {
        printf("Bit %d: ", i + 1);
        scanf("%d", &data[i]);
    }

    // Calculate parity bit (for even parity)
    int parity = calculateParity(data, size);
    printf("Calculated Parity Bit (Even Parity): %d\n", parity);
}
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
// Append the parity bit to the data for transmission
int transmittedData[9];
for (int i = 0; i < size; i++) {
    transmittedData[i] = data[i];
}
transmittedData[size] = parity;

// Simulate reception and check parity at the receiver's end
int receivedParity = calculateParity(transmittedData, size + 1);
if (receivedParity == 0) {
    printf("No error detected. Data received correctly.\n");
} else {
    printf("Error detected in the received data.\n");
}

return 0;
}
```

Output

Enter 8-bit data (1 or 0 for each bit):

Bit 1: 1

Bit 2: 0

Bit 3: 1

Bit 4: 1

Bit 5: 0

Bit 6: 0

Bit 7: 1

Bit 8: 0

Calculated Parity Bit (Even Parity): 0

No error detected. Data received correctly.

2. Check sum

The problem with the parity bit method is that it reliably detects only one-bit errors in the message. If the message contains a burst error, the parity bit method will not work. Therefore, the parity bit is only used when cables are challenging for error detection, as those cables have very low error rates. To solve the problem of the parity bit method, the checksum method is used.

- The checksum is the complement of the total of all the code-words. The Checksum is placed at the end of the message. The checksum method operates on words instead of bits.

Example

```
#include <stdio.h>
```

```
// Function to calculate 1's complement sum of two 8-bit numbers
unsigned char onesComplementSum(unsigned char a, unsigned char b) {
    unsigned short sum = a + b;
    if (sum > 0xFF) { // Check for carry
        sum = (sum & 0xFF) + 1;
    }
    return (unsigned char)sum;
}
```

```
// Function to calculate the checksum
unsigned char calculateChecksum(unsigned char data[], int size) {
    unsigned char checksum = data[0];
    for (int i = 1; i < size; i++) {
        checksum = onesComplementSum(checksum, data[i]);
    }
    return ~checksum; // Return the complement of the sum
}
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
}

// Function to validate the checksum at the receiver's end
int validateChecksum(unsigned char data[], int size, unsigned char receivedChecksum) {
    unsigned char sum = data[0];
    for (int i = 1; i < size; i++) {
        sum = onesComplementSum(sum, data[i]);
    }
    sum = onesComplementSum(sum, receivedChecksum);
    return sum == 0xFF; // If sum is 0xFF, no error detected
}

// Function to convert binary string to unsigned char
unsigned char binaryStringToChar(char *binaryStr) {
    unsigned char value = 0;
    for (int i = 0; i < 8; i++) {
        value = (value << 1) | (binaryStr[i] - '0');
    }
    return value;
}

// Function to print an 8-bit number in binary format
void printBinary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main() {
    unsigned char data[4];
    int size = 4;
    char binaryStr[9]; // To store 8-bit binary string input

    // Input 4 frames of 8-bit data from the user in binary
    printf("Enter 4 frames of 8-bit data (in binary, e.g., 11001100):\n");
    for (int i = 0; i < size; i++) {
        printf("Frame %d: ", i + 1);
        scanf("%8s", binaryStr);
        data[i] = binaryStringToChar(binaryStr);
    }

    // Calculate the checksum at the sender's side
    unsigned char checksum = calculateChecksum(data, size);
    printf("Calculated Checksum (in binary): ");
    printBinary(checksum);

    // Simulate sending and receiving the data along with checksum
    printf("\nSimulating reception of data...\n");

    // Output received data frames and checksum
    printf("Received Data Frames:\n");
    for (int i = 0; i < size; i++) {
        printBinary(data[i]);
    }
}
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
printf("Received Checksum: ");
printBinary(checksum);

// Validate the checksum at the receiver's end
if (validateChecksum(data, size, checksum)) {
    printf("No error detected. Data received correctly.\n");
} else {
    printf("Error detected in the received data.\n");
}

return 0;
}
```

Enter 4 frames of 8-bit data (in binary, e.g., 11001100):

Frame 1: 11001100

Frame 2: 10101010

Frame 3: 11110000

Frame 4: 11000011

Calculated Checksum (in binary): 11010011

Simulating reception of data...

Received Data Frames:

11001100

10101010

11110000

11000011

Received Checksum: 11010011

No error detected. Data received correctly.

With error inputs

Frame 1: 11001100 Frame 2: 10101010 Frame 3: 11110000 Frame 4: 11000011

Will get error

Error Correction

The error detection method is used to check whether an error has occurred in the data. Now it's time to learn how we can fix the detected errors using error correction methods. The error correction method is used to find out the exact number of corrupted bits to correct.

Error correction can be done in two ways:

1. Backward Error Correction
2. Forward Error Correction
 - **Backward Error Correction:** In simple words, the receiver sends an acknowledgment to the sender when corrupt data is received, and the sender retransmits the data to the receiver.
 - **Forward Error Correction:** When the receiver receives data that contains errors, the receiver attempts to retrieve and correct the data using methods. The 4 main methods of Forward-Error correction are as follows:
 1. Hamming Codes
 2. Binary Convolution Codes
 3. Reed-Solomon Codes
 4. Low-Density Parity-Check Codes

3. Hamming codes

Example Code:

```
#include <stdio.h>
```

```
// Function to calculate the parity bits
void calculateHamming(int data[], int hammingCode[]) {
    // Data bits
    hammingCode[2] = data[0];
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
hammingCode[4] = data[1];
hammingCode[5] = data[2];
hammingCode[6] = data[3];

// Parity bits
hammingCode[0] = hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
hammingCode[1] = hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
hammingCode[3] = hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];
}

// Function to check and correct errors
int checkHamming(int hammingCode[]) {
    int c1 = hammingCode[0] ^ hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
    int c2 = hammingCode[1] ^ hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
    int c4 = hammingCode[3] ^ hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];

    int errorPos = c1 + (c2 * 2) + (c4 * 4);

    return errorPos;
}

int main() {
    int data[] = {1, 0, 1, 1}; // 4-bit data
    int hammingCode[7];

    // Generate Hamming Code
    calculateHamming(data, hammingCode);

    // Print Hamming Code
    printf("Hamming Code: ");
    for (int i = 0; i < 7; i++) {
        printf("%d", hammingCode[i]);
    }
    printf("\n");

    // Simulate a single-bit error in transmission
    hammingCode[2] = 0; // Intentionally flipping a bit

    // Detect and correct error
    int errorPos = checkHamming(hammingCode);
    if (errorPos == 0) {
        printf("No error detected.\n");
    } else {
        printf("Error detected at position: %d\n", errorPos);
        hammingCode[errorPos - 1] = !hammingCode[errorPos - 1];
        printf("Corrected Hamming Code: ");
        for (int i = 0; i < 7; i++) {
            printf("%d", hammingCode[i]);
        }
        printf("\n");
    }

    return 0;
}
```

Output

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

Hamming Code: 0110011

Error detected at position: 3

Corrected Hamming Code: 0110011

4. Flow control Mechanisms

Flow control is a critical mechanism in computer networks that ensures that the sender does not overwhelm the receiver by sending data faster than the receiver can process. It helps maintain data integrity, prevents packet loss, and ensures efficient utilization of network resources. Two common flow control mechanisms are used in computer networks:

1. Stop-and-Wait Protocol

- **Concept:** The sender transmits one frame or packet of data and then waits for an acknowledgment from the receiver before sending the next packet.
- **Advantages:**
 - Simple and easy to implement.
 - Avoids overwhelming the receiver.
- **Disadvantages:**
 - Inefficient use of bandwidth, especially in long-distance networks or high-speed networks.
 - The sender remains idle while waiting for acknowledgment, leading to underutilization of the network's capacity.

2. Sliding Window Protocol

- **Concept:** The sender can send multiple frames before needing an acknowledgment, governed by a "window" size. The receiver acknowledges packets in groups, allowing for continuous data transmission without waiting for each acknowledgment.
- **Key Features:**
 - The window size controls the amount of unacknowledged data in the network at any given time.
 - As the receiver acknowledges data, the window "slides," allowing the sender to transmit more frames.
- **Types:**
 - **Go-Back-N:** If an error is detected in a frame, the receiver discards all subsequent frames, and the sender must retransmit all frames from the erroneous one onwards.
 - **Selective Repeat:** The receiver only requests retransmission of the erroneous frames, while successfully received frames are buffered, providing better efficiency compared to Go-Back-N.
- **Advantages:**
 - Makes better use of available bandwidth compared to Stop-and-Wait.
 - Can handle errors more efficiently, especially with the Selective Repeat variant.
- **Disadvantages:**
 - More complex to implement.
 - Requires more buffer space at both the sender and receiver.

3. Credit-Based Flow Control

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

- **Concept:** The receiver provides the sender with "credits" or tokens that determine how many frames can be sent without receiving an acknowledgment. Each credit represents a unit of buffer space at the receiver's end.
- **Advantages:**
 - Ensures that the sender does not send more data than the receiver can handle.
 - Adapts dynamically to changes in network conditions and receiver processing speeds.
- **Disadvantages:**
 - Additional overhead to maintain and manage the credit system.

4. Rate-Based Flow Control

- **Concept:** In this mechanism, the sender is regulated to transmit data at a rate that the network can handle, which is often controlled by the receiver or an intermediate network device.
- **Advantages:**
 - Ensures smooth data flow, preventing congestion and packet loss.
- **Disadvantages:**
 - May require more complex calculations to dynamically adjust the rate.
 - Suitable for environments with stable traffic patterns but may not perform well under sudden bursts of data.

Importance of Flow Control in Computer Networks

- **Prevents Data Loss:** Ensures that the receiver can handle the incoming data without dropping packets.
- **Efficient Bandwidth Usage:** By pacing data transmission, flow control mechanisms optimize the use of available network bandwidth.
- **Prevents Congestion:** It helps avoid network congestion by controlling the rate of data transmission.
- **Ensures Data Integrity:** Avoids situations where data is lost or corrupted due to buffer overflow at the receiver.

Flow control is often implemented in conjunction with congestion control mechanisms, particularly in protocols like TCP, to maintain a balance between transmission rate and network conditions.

Stop and wait Protocol (c++ code)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
```

```
using namespace std;
```

```
// Simulates the Stop-and-Wait protocol for flow control
void stopAndWait(int totalPackets) {
    for (int packet = 1; packet <= totalPackets; ++packet) {
        cout << "Sending packet " << packet << "..." << endl;
```

```
        // Simulate random acknowledgment loss or delay
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
int ack = rand() % 2;

// If ack = 1, the acknowledgment is received; else, it's lost.
if (ack == 1) {
    // Simulate transmission delay
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Acknowledgment received for packet " << packet << "." << endl;
} else {
    // Simulate lost acknowledgment; retransmit the packet
    cout << "Acknowledgment lost for packet " << packet << ". Retransmitting..." <<
endl;
    packet--; // Retransmit the packet
}

// Simulate delay between packet transmissions
this_thread::sleep_for(chrono::seconds(1));
}
}

int main() {
    srand(time(0)); // Initialize random seed for acknowledgment loss simulation

    int totalPackets;

    // Input: Total number of packets to send
    cout << "Enter the total number of packets to send: ";
    cin >> totalPackets;

    // Simulate the Stop-and-Wait protocol
    stopAndWait(totalPackets);

    return 0;
}
```

Explanation:

- **stopAndWait** function:
 - Simulates the transmission of a packet from sender to receiver.
 - Uses random numbers to simulate packet acknowledgment (either received or lost).
 - If the acknowledgment is lost, the packet is retransmitted.
- **this_thread::sleep_for**: Simulates the delay between transmission and acknowledgment.

Sample Input/Output:

Enter the total number of packets to send: 5

Possible Output:

```
Sending packet 1...
Acknowledgment received for packet 1.
Sending packet 2...
Acknowledgment received for packet 2.
Sending packet 3...
```


CN LAB EXERCISES
SUBJECT CODE: BCSE308P

Acknowledgment lost for packet 3. Retransmitting...

Sending packet 3...

Acknowledgment received for packet 3.

Sending packet 4...

Acknowledgment received for packet 4.

Sending packet 5...

Acknowledgment lost for packet 5. Retransmitting...

Sending packet 5...

Acknowledgment received for packet 5.

In this output, the acknowledgment for packet 3 and packet 5 was lost, leading to retransmission of those packets. The behavior may change each time the program is run due to the randomness of acknowledgment loss simulation.

Sliding Window Protocol (Go-Back-N)

```
#include <iostream>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <thread>
```

```
#include <chrono>
```

```
using namespace std;
```

```
void goBackN(int totalPackets, int windowSize) {
```

```
    int nextPacket = 1;
```

```
    while (nextPacket <= totalPackets) {
```

```
        for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
```

```
            cout << "Sending packet " << nextPacket + i << "..." << endl;
```

```
        }
```

```
        for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
```

```
            int ack = rand() % 2;
```

```
            if (ack == 1) {
```

```
                this_thread::sleep_for(chrono::seconds(1));
```

```
                cout << "Acknowledgment received for packet " << nextPacket + i << "." << endl;
```

```
            } else {
```

```
                cout << "Acknowledgment lost for packet " << nextPacket + i << ". Retransmitting  
window..." << endl;
```

```
                break;
```

```
            }
```

```
        }
```

```
        nextPacket += windowSize;
```

```
    }
```

```
}
```

```
int main() {
```

```
    srand(time(0)); // Initialize random seed
```

```
    int totalPackets, windowSize;
```

```
    cout << "Enter the total number of packets to send: ";
```

```
    cin >> totalPackets;
```

```
    cout << "Enter the window size: ";
```

```
    cin >> windowSize;
```

```
    goBackN(totalPackets, windowSize);
```

```
    return 0;
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

}

Sample Input/Output for Go-Back-N:

Input:

Enter the total number of packets to send: 5

Enter the window size: 3

Output:

Sending packet 1...

Sending packet 2...

Sending packet 3...

Acknowledgment received for packet 1.

Acknowledgment lost for packet 2. Retransmitting window...

Sending packet 2...

Sending packet 3...

Sending packet 4...

Acknowledgment received for packet 2.

Acknowledgment received for packet 3.

Acknowledgment received for packet 4.

Sliding Window Protocol (Selective Repeat)

```
#include <iostream>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <thread>
```

```
#include <chrono>
```

```
using namespace std;
```

```
void selectiveRepeat(int totalPackets, int windowSize) {
```

```
    bool acks[totalPackets + 1] = {false}; // Array to track acknowledgments
```

```
    int sentPackets = 0;
```

```
    while (sentPackets < totalPackets) {
```

```
        for (int i = 0; i < windowSize && sentPackets + i < totalPackets; ++i) {
```

```
            if (!acks[sentPackets + i]) {
```

```
                cout << "Sending packet " << sentPackets + i + 1 << "..." << endl;
```

```
            }
```

```
        }
```

```
        for (int i = 0; i < windowSize && sentPackets + i < totalPackets; ++i) {
```

```
            if (!acks[sentPackets + i]) {
```

```
                int ack = rand() % 2;
```

```
                if (ack == 1) {
```

```
                    this_thread::sleep_for(chrono::seconds(1));
```

```
                    cout << "Acknowledgment received for packet " << sentPackets + i + 1 << ".
```

```
<< endl;
```

```
                    acks[sentPackets + i] = true;
```

```
                } else {
```

```
                    cout << "Acknowledgment lost for packet " << sentPackets + i + 1 << ".
```

```
Retransmitting..." << endl;
```

```
                }
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
    }  
  }  
  
  for (int i = 0; i < totalPackets; ++i) {  
    if (acks[i]) {  
      ++sentPackets;  
    }  
  }  
}  
}  
  
int main() {  
  srand(time(0)); // Initialize random seed  
  int totalPackets, windowSize;  
  cout << "Enter the total number of packets to send: ";  
  cin >> totalPackets;  
  cout << "Enter the window size: ";  
  cin >> windowSize;  
  selectiveRepeat(totalPackets, windowSize);  
  return 0;  
}
```

Input:

Enter the total number of packets to send: 5
Enter the window size: 3

Output:

Sending packet 1...
Sending packet 2...
Sending packet 3...
Acknowledgment received for packet 1.
Acknowledgment lost for packet 2. Retransmitting...
Acknowledgment received for packet 3.
Sending packet 2...
Acknowledgment received for packet 2.

Credit-Based Flow Control

```
#include <iostream>  
#include <thread>  
#include <chrono>  
using namespace std;  
void creditBasedFlowControl(int totalPackets, int initialCredits) {  
  int credits = initialCredits;  
  
  for (int packet = 1; packet <= totalPackets; ++packet) {  
    if (credits > 0) {  
      cout << "Sending packet " << packet << " with " << credits << " credits left." <<  
endl;  
      credits--; // Decrease credits as we send packets  
      this_thread::sleep_for(chrono::seconds(1));  
    } else {  
      cout << "No credits left. Waiting for credit replenishment..." << endl;  
    }  
  }  
}
```

CN LAB EXERCISES
SUBJECT CODE: BCSE308P

```
        this_thread::sleep_for(chrono::seconds(2)); // Simulate replenishment delay
        credits = initialCredits; // Replenish credits
    }
    cout << "Acknowledgment received for packet " << packet << "." << endl;
}
}
int main() {
    int totalPackets, initialCredits;
    cout << "Enter the total number of packets to send: ";
    cin >> totalPackets;
    cout << "Enter the initial number of credits: ";
    cin >> initialCredits;
    creditBasedFlowControl(totalPackets, initialCredits);
    return 0;
}
```

Sample Input/Output for Credit-Based Flow Control:

Input:

Enter the total number of packets to send: 5
Enter the initial number of credits: 2

Output:

Sending packet 1 with 2 credits left.
Acknowledgment received for packet 1.
Sending packet 2 with 1 credits left.
Acknowledgment received for packet 2.
No credits left. Waiting for credit replenishment...
Sending packet 3 with 2 credits left.
Acknowledgment received for packet 3.
Sending packet 4 with 1 credits left.
Acknowledgment received for packet 4.
No credits left. Waiting for credit replenishment...
Sending packet 5 with 2 credits left.
Acknowledgment received for packet 5.

Key Takeaways:

- Each of the above flow control mechanisms (Stop-and-Wait, Go-Back-N, Selective Repeat, and Credit-Based Flow Control) demonstrates different approaches for managing data transmission between sender and receiver in a network.